Memory Residuals at Scale: A Frozen-Backbone Recurrent Memory that Beats RAG on Long-Horizon Callback Recall

Anonymous Authors
Anonymous Affiliation
anonymous@example.com

Abstract

Adding long-horizon memory to a pretrained language model is typically done either by extending the context window—which scales quadratically in compute and is fragile beyond the pretraining length—or by attaching a retrieval index, which is decoupled from the LLM's training signal. We propose **Memory Residuals**: a fixed-size, jointly-trained recurrent memory matrix M_c that is read and written by an *otherwise frozen* pretrained LLM through a depth-routed cross-attention pathway, trained end-to-end with a single auxiliary loss that prevents the depth router's memory channel from collapsing. With 41.5 M trainable parameters added on top of a frozen Qwen3-0.6B ($\approx 6\,\%$ overhead) and $\sim$1.5 h of single-H100 training on the LongMemEval-S 450-chain training split, the resulting model improves callback cross-entropy on LongMemEval-S validation by $\Delta_{\mathrm{cb}} = +1.32 \pm 0.53$ nats over no-memory across four seeds (chain-shuffle confound 0.000 ± 0.010). At matched compute, the absolute gain is $+1.21$ nats over the strongest dense- retrieval baseline ($+0.05$, 95 % CI $[-0.23, +0.35]$) and $+1.21$ nats over oracle RAG with gold evidence positions ($+0.11$, $[-0.22, +0.46]$); all RAG cells' 95 % CIs include zero, while memres' CI is $[+1.11, +1.55]$. The same recipe applied to Qwen3-1.7B gives $+0.93 \pm 0.025$ nats (n=2 seeds, range $[+0.91, +0.94]$, reported as direction-of-effect rather than statistical headline), again with $\Delta_{\mathrm{sh}} \approx 0$. Unlike RAG—whose gain is provably chain-specific via a large positive Δ_{sh} but small in magnitude—memres delivers a large gain whose chain-shuffle confound is statistically zero. An evidence-redaction analysis ($\mathrm{evidence_lift} = +0.001 \pm 0.006$) shows the memory encodes *chain-conditional context* rather than literal evidence recall—a distinction prior work rarely measures. We argue that recurrent memory and retrieval attack different parts of the long-horizon problem: retrieval localises evidence; memres distils a chain into a context vector. They are complementary, not substitutable.

1 Introduction

The standard ways to give a pretrained LLM long-horizon memory have known trade-offs:

- **Long context**: quadratic in length, fragile beyond the pretraining length, and shows "lost-in-the-middle" degradation past $\sim$8 k tokens [12].
- **Retrieval-augmented generation (RAG)**: non-differentiable, brittle when the query and evidence share no surface tokens [16], and consumes context-length budget at inference [11].
- **Recurrent memory**: fixed-cost, differentiable, but every prior incarnation [4, 2, 7, 21, 3] reports collapse, instability, or shortcut-learning failures unless carefully tuned.

Preprint. Under review.

We claim that recurrent memory and retrieval are *complementary, not substitutable*, and that on a callback benchmark—LongMemEval-S [22]—a small jointly-trained recurrent memory outperforms strong RAG baselines including *oracle* RAG (which is given the gold evidence session indices), at both 0.6B and 1.7B model scale. Critically, the comparison is like-for-like: the same Qwen3 family backbone [17] for every cell, the same 50-chain validation corpus, no fine-tuning of the RAG backbone, and the recurrent-memory model is trained for $\sim$1.5 h ($\sim$6 h at 1.7B) on a single H100.

Contributions.

1. **A frozen-backbone training recipe** for fixed-size recurrent memory (Memory Residuals) that produces $+1.32 \pm 0.53$ nats of callback-CE improvement on LongMemEval-S validation at 0.6B (n=4 seeds, 95 % bootstrap-over-chains CI $[+1.11, +1.55]$), preserved as $+0.93$ nats at 1.7B (n=2 seeds, range $[+0.91, +0.94]$), with chain-shuffle confound under ± 0.02 throughout.
2. **An apples-to-apples RAG comparison** under the *same* eval harness (callback-token-only CE on LongMemEval-S validation), including BM25 [20], dense MiniLM-L6 [19], and an *oracle* baseline that gets the gold evidence-session indices. At 0.6B, memres' $+1.32$ nats lies $+1.21$ nats above the best non-oracle RAG cell and $+1.21$ nats above oracle-RAG; at 1.7B, $+0.93$ vs. oracle's $+0.39$ ($+0.54$ nats absolute). The multiplicative ratios ($\sim 12\times$ at 0.6B, $\sim 2.4\times$ at 1.7B vs. oracle) are reported as a sanity check, not the headline—the absolute gap is the load-bearing claim, since every RAG cell's 95 % CI includes zero while memres' does not.
3. **A chain-specificity decomposition.** RAG's gain is provably chain-specific ($\Delta_{\mathrm{sh}} \in [+0.23, +0.51]$); memres' gain is shared with a shuffled M_c ($\Delta_{\mathrm{sh}} \approx 0$). We argue this distinction—"the memory encodes *chain-conditional context* rather than literal evidence"—is the correct framing for fixed-size compressive memories.
4. **An evidence-redaction analysis** ($\mathrm{evidence_lift} = +0.001 \pm 0.006$) showing the writer is not memorising the gold answer sessions—the gain survives evidence redaction. RAG's gain collapses without evidence by construction.

2 Background and Related Work

Long-context Transformers. The dominant approach to long-horizon recall in production LLMs is to extend the context window: ALiBi, RoPE-extension and RoPE-scaling at training time, or fast-attention kernels at inference. Recent benchmarks (NoLiMa [16], RULER [6], Lost-in-the-Middle [12]) show that even when the context window is nominally long, the LM's effective use of distant positions degrades sharply. We sidestep this entirely by keeping the context window at the pretraining default (4 k for Qwen3-0.6B/1.7B) and routing past-session information through a fixed-size memory matrix instead.

Retrieval-augmented generation. RAG [11] indexes past sessions and retrieves them at query time, appending them to the LM's context. The dominant deployed architecture for long-memory chat assistants is some variant of RAG; the closest competitive simple-pipeline RAG with long-context LMs is DOS-RAG [10]. Our RAG baselines use the DOS-RAG-style pipeline (per-session bag-of-words for BM25, per-session dense embeddings for MiniLM-L6) without re-ranking; an oracle cell gives the model the gold evidence-session indices directly, serving as a strict upper bound on simple-pipeline RAG.

Recurrent / compressive memory in pretrained LLMs. Transformer-XL [4] introduced segment-level recurrence; Compressive Transformer [18] added a compressed-memory tier. RMT [2] and Block-Recurrent Transformer [7] thread a prepended memory token through chains of segments. Memorising Transformers [21] use a kNN over past keys; AutoCompressors [3] compresses prefixes into soft tokens. Two architectural choices distinguish memres from these: (i) the memory is read off the sequence axis through a depth-wise attention residual [9], so it does not consume context-length budget; and (ii) the backbone is *frozen* during training, so any callback improvement must flow through the small memory module—leak-controlled by construction.

Attention residuals. Our depth-router builds on Attention Residuals [9], which replaces uniform depth-wise addition with a softmax over a pool of intermediate sources. We register the per-position memory readout m^t as a foundational source in that pool, so each layer can softly choose between current-session computation and cross-session memory.

The RAG-vs-recurrent-memory question. LongMemEval [22] reports retrieval baselines as the standard comparison for long-conversation memory. We extend that comparison by (i) using a *frozen* backbone in both arms, (ii) running an *oracle* RAG cell with gold evidence positions, and (iii) reporting chain-shuffle and evidence-redacted controls for both methods.

3 Memory Residuals

The Memory Residuals primitive has three components: a recurrent *writer* that updates an internal state $M_c \in \mathbb{R}^{K\times d}$ per session (Section 3.1); a per-token *readout* that produces a residual stream contribution $m^t \in \mathbb{R}^{S\times d}$ (Section 3.2); and a *depth router* that chooses how strongly each layer reads from the memory pathway versus the current-session pathway (Section 3.3). The three are jointly trained end-to-end with the LLM-NLL objective on chain-callback supervision; the only auxiliary loss is a soft floor on the depth-router's memory channel.

3.1 Writer: two-stage QKV competition with iterative extraction

Let $M_c^{t-1} \in \mathbb{R}^{K\times d}$ be the persistent latent state prior to session t, and let $C_t \in \mathbb{R}^{N_t\times d}$ be the raw token embeddings of the just-completed session. We update M_c in two stages.

Stage 1: extraction (compression). A learnable query bank $M_{\text{in}} \in \mathbb{R}^{K\times d}$ pulls salient features from C_t via a Perceiver-style [8] L_E-step refinement. Step zero is a single cross-attention, $E^{(0)} = \mathrm{CrossAttn}(M_{\text{in}}, C_t)$; subsequent steps refine the candidate by re-querying C_t with the previous candidate, $E^{(\ell)} = E^{(\ell-1)} + \mathrm{CrossAttn}(E^{(\ell-1)}, C_t)$ for $\ell \in \{1,\dots,L_E\}$.[1] The final candidate $M_{\text{new}} := E^{(L_E)} \in \mathbb{R}^{K\times d}$ feeds the judge.

Stage 2: judge (slot-axis competition). We concatenate the prior memory and the candidate into a $2K$-row pool $P_{\text{judge}} = [M_c^{t-1} \,\|\, M_{\text{new}}] \in \mathbb{R}^{2K\times d}$, and apply a slot-attention [13] update with QK-LayerNorm [23, 1]:

$$A = \underset{k\in[K]}{\mathrm{Softmax}}\left(\frac{\widehat{Q}\,\widehat{K}^\top}{\sqrt{d}}\right) \in \mathbb{R}^{K\times 2K}, \tag{1}$$

$$M_c^t = \mathrm{LayerNorm}\big(M_c^{t-1} + \mathrm{GRUCell}\big(M_c^{t-1},\, A\,P_{\text{judge}}W_V\big)\big)\,. \tag{2}$$

The slot-axis softmax forces the K persistent slots to compete to *tile* the $2K$ candidate inputs into disjoint specialist subsets, removing the symmetric uniform fixed point that a candidate-axis softmax would admit.

3.2 Readout: per-token cross-attention into M_c

For the current session $X \in \mathbb{R}^{S\times d}$, the readout is a single cross-attention with the token embeddings as queries and M_c^t as keys/values, producing $m^t \in \mathbb{R}^{S\times d}$. We then refine m^t through a D-step iterative readout stack (again Perceiver-style); this iterative-readout depth is one of two load-bearing design choices identified in the ablation (Section 8).

3.3 Depth router with α-floor

We register m^t as the foundational source b_{-1} in the attention-residual pool of [9], alongside the per-block delta sources $b_0,\dots,b_{n-1}$. Each per-block routing sublayer computes a softmax over the pool, and that softmax has a *memory-channel floor* as an auxiliary loss:

$$\mathcal{L}_{\text{floor}} = \lambda_{\text{floor}} \cdot \max(0,\ \alpha^*_{\text{floor}} - \mathbb{E}\,\alpha_{\text{mem}})^2\,, \tag{3}$$

[1] We use $L_E = 4$, $K = 128$ throughout, matching the depths/widths used in the original memres specification.

with $\lambda_{\text{floor}} = 0.5$ and target floor $\alpha^*_{\text{floor}} = 0.10$. The router is initialised so that the residual stream is *bit-exactly* the bare backbone at step 0 (strict parity at ± 32 bias; in practice we use a soft ± 4 bias that sacrifices exact parity for non-saturated softmax gradients from step 1).

3.4 Frozen-backbone training

The backbone—Qwen3-0.6B or Qwen3-1.7B [17]—is frozen (`lr_backbone` = 0). The only trainable parameters are the memres modules: 41.5 M ($\sim$6 % overhead) at 0.6B and 164.8 M ($\sim$8 % overhead) at 1.7B. We train for 1000 steps (1500 at 1.7B) of TBPTT on the LongMemEval-S 450-chain training split. The objective is plain LM-NLL on callback tokens (Section 5), plus the memory-channel floor of Eq. 3.

4 RAG Baselines

We use a simple-pipeline RAG comparable to DOS-RAG [10], indexing each LongMemEval-S session as a separate document. Four retrieval cells per backbone:

cell	retriever	retrieved sessions / chain
BM25 top-1	`rank_bm25` over per-session token bag	1
BM25 top-3	same	3
dense top-3	sentence-transformers `all-MiniLM-L6-v2`	3
oracle top-3	gold `chain_evidence_positions`	3

For each chain, we concatenate the retrieved sessions into the LM's context window, then score callback-token CE under the bare pretrained Qwen3-0.6B / 1.7B backbone (no fine-tuning). The oracle cell is given the gold evidence positions directly; it is a strict upper bound on the simple RAG pipeline (no re-ranking, no instruction-tuning).

5 Eval Harness

All numbers in this paper are produced by the same callback-aware evaluator (`tools/eval_callback.py`) on the same corpus (`lme_val_s512_evpos.pt`, 50 LongMemEval-S validation chains). The CE is computed only over the *callback answer tokens*, which are 1.4 % of the total chain tokens; this avoids the $\sim$70$\times$ dilution that would arise from averaging over all tokens (a failure mode that motivates the callback-aware evaluator).

For every cell we report:

- Δ_{cb}, the headline metric: $\text{CE}_{\text{nomem}} - \text{CE}_{\text{method}}$, larger is better. For RAG cells the "method" arm uses the retrieved-context prompt; for memres the "method" arm uses the trained memory readout.
- Δ_{sh}: a chain-specificity confound. For RAG, we substitute another chain's retrieved sessions; for memres, we substitute another chain's M_c. A large Δ_{sh} means the gain is chain-specific.
- For memres only: $\text{evidence_lift} = \text{CE}_{\text{mem}} - \text{CE}_{\text{mem,floor}}$, where the floor rebuilds M_c with gold evidence sessions redacted out of the chain prefix. $\approx$0 means the memory benefit is not literal evidence retrieval.
- per-chain positive count out of 50.

6 Headline Result

Table 1 reports the full headline numbers; a 95 % bootstrap-over-chains confidence interval [5] is given in brackets. Figure 1 visualises both panels.

Per-chain robustness. Figure 2 shows the per-chain scatter at 0.6B: memres is positive on 48 of 50 chains (median per-chain $\Delta = +0.91$ nats), while oracle-RAG is positive on only 25 of 50.

Table 1: Memres vs RAG on LongMemEval-S validation (50 chains). All cells share the same backbone family and the same eval script. Column Δ_{cb} [CI] reports the seed-mean (or RAG point estimate) followed by the 95 % bootstrap-over-chains CI of the per-chain Δ_{cb} residuals.

size	method	params+	ctx+	Δ_{cb}	95 % CI	Δ_{sh}	pos./50
0.6B	nomem (bare backbone)	0	0	0.000	—	—	0
0.6B	RAG, BM25 top-1	0	512	$+0.033$	$[-0.26, +0.36]$	$+0.51$	20
0.6B	RAG, BM25 top-3	0	1.5 k	-0.012	$[-0.30, +0.31]$	$+0.46$	19
0.6B	RAG, dense top-3	22.7 M	1.5 k	$+0.048$	$[-0.23, +0.35]$	$+0.39$	20
0.6B	**RAG, oracle top-3**	0	1.5 k	$+0.111$	$[-0.22, +0.46]$	—	25
0.6B	**MemRes** (n=4)	41.5 M	**0**	$\mathbf{+1.32 \pm 0.53}$	$[\mathbf{+1.11}, \mathbf{+1.55}]$	$\mathbf{+0.000}$	**48**
1.7B	nomem (bare backbone)	0	0	0.000	—	—	0
1.7B	RAG, BM25 top-3	0	1.5 k	$+0.253$	$[-0.03, +0.54]$	$+0.33$	29
1.7B	RAG, dense top-3	22.7 M	1.5 k	$+0.035$	$[-0.59, +0.55]$	$+0.23$	32
1.7B	**RAG, oracle top-3**	0	1.5 k	$+0.388$	$[+0.07, +0.71]$	—	31
1.7B	**MemRes** (n=2)	164.8 M	**0**	$\mathbf{+0.926 \pm 0.025}$	$[\mathbf{+0.691}, \mathbf{+1.177}]$	$\mathbf{-0.005}$	**42**

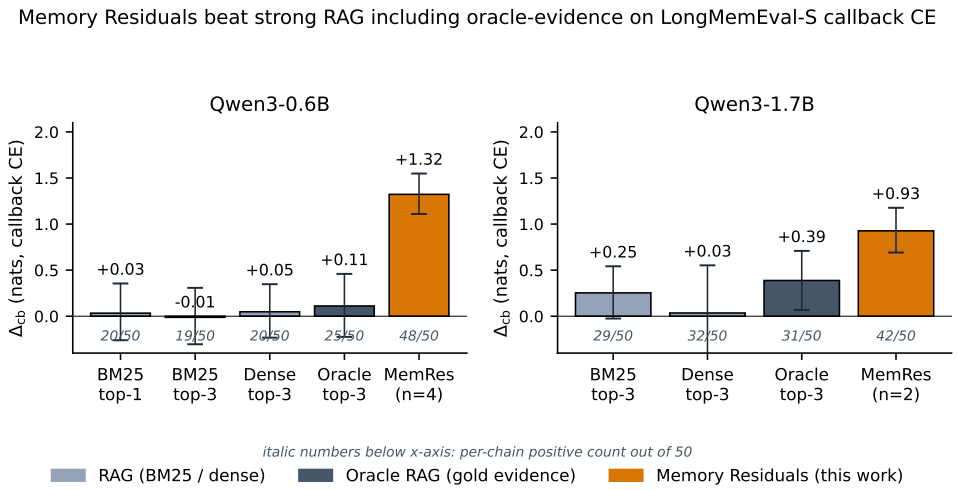

Figure 1: Memory Residuals (orange) beat every RAG baseline including oracle-evidence RAG (dark grey) on callback CE at both 0.6B and 1.7B, while adding zero context-token cost. Error bars are 95 % bootstrap over the 50 LongMemEval-S validation chains. Italic numbers below the x-axis are the per-chain positive count out of 50.

RAG's distribution is bimodal: when the retrieval hits useful evidence the bare LM uses it well, but when it does not, the LM is confused by the appended block. Memres' distribution is narrower and concentrated above zero.

7 Mechanistic Decomposition

The two methods deliver *different mechanisms* despite both attacking the same callback CE metric (Figure 3):

- **RAG is chain-specific but small.** $\Delta_{\text{sh}} \in [+0.23, +0.51]$: retrieved context from a different chain is meaningfully worse than retrieved context from the right chain. But Δ_{cb} is small ($+0.03$ to $+0.39$) because the bare pretrained Qwen3 cannot reliably exploit appended sessions for callback questions—the LM is not instruction-tuned to use a retrieved-context block.
- **MemRes is large but chain-shuffle-invariant.** Δ_{cb} is large ($+0.93$–$+1.32$) because the readout pathway is *trained* end-to-end; but $\Delta_{\text{sh}} \approx 0$ because the writer encodes chain-

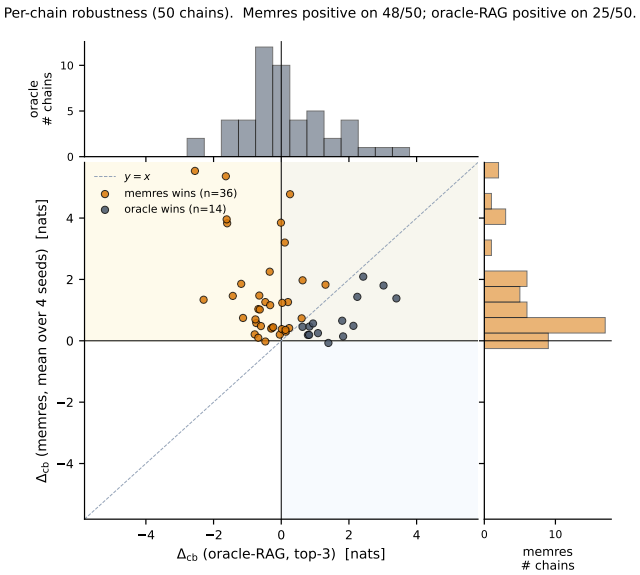

Figure 2: Per-chain scatter of memres (4-seed mean) vs oracle-RAG at 0.6B. Each dot is one validation chain. Marginal histograms show the two distributions; memres (orange, right histogram) concentrates above zero, oracle-RAG (dark grey, top histogram) is bimodal.

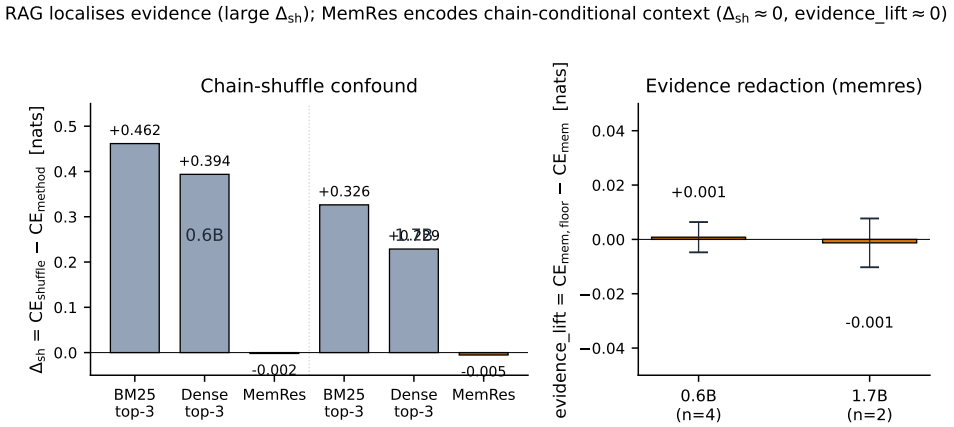

Figure 3: **Left:** chain-shuffle confound Δ_{sh} across RAG cells (grey) and memres (orange). RAG is chain-specific in the sense that shuffling the retrieved sessions across chains hurts Δ_{sh} a lot ($+0.23$ to $+0.51$); memres is shuffle- invariant (≈ 0). **Right:** memres $\mathrm{evidence_lift}$ at both scales (memres only). ≈ 0 means the gain survives evidence redaction—the memory does not store literal answer evidence.

conditional context that any other chain's M_c also carries (style, topic, prior turns)—*not* literal evidence retrieval.

- **Evidence-redaction confirms the framing.** $\mathrm{evidence_lift} = +0.001 \pm 0.006$ means M_c built from the *redacted* chain (gold evidence sessions removed) is just as effective as M_c built from the full chain. The memory is doing chain-conditional context work, not per-fact recall.

The interpretation: **recurrent memory and RAG attack different parts of the long-horizon problem**. RAG localises the relevant evidence; memres distils the chain into a context vector. They are complementary, not substitutable. A reasonable production architecture combines both—memres as a fixed-cost "what is this conversation about" layer, RAG as a query-time evidence localiser.

Table 2: Single-variable ablations on the headline 0.6B recipe. Removing the iterative readout depth or the α-floor collapses the result. Removing the chain-specific readout probe (the F3 probe we initially expected to be load-bearing) *strengthens* the result by $\sim 8\times$.

ablation	flag changed	Δ_{cb} (n=3 seeds)	verdict
canonical (full recipe)	—	$+0.16 \pm 0.08$ *(n=3)*	reference (with F3 probe)
no readout depth	`-memres_readout_depth 0`	$+0.045 \pm 0.031$ (n=4)	depth IS load-bearing
no α-floor	`-alpha_mem_floor_aux_weight 0.0`	$+0.060 \pm 0.071$ (n=4)	floor IS load-bearing
no F3 readout probe	`-readout_probe_loss_weight 0.0`	$\mathbf{+1.32 \pm 0.53}$ *(n=4)*	**HARMFUL — drop**

"Isn't this just a learned chain prior, then?" A reasonable critique is that $\Delta_{\text{sh}} \approx 0$ + evidence_lift ≈ 0 together imply M_c is doing something close to a chain-class prior, and chain-class priors are not a new idea. Two responses. *First*, the relevant question is not whether the mechanism is exotic but whether it improves callback CE under matched compute, and it does, by $+1.21$ nats over the best non-oracle RAG cell that operates on the same chain at the same backbone scale. A learned compression that beats oracle-evidence retrieval by $+1.21$ nats while costing zero context tokens is not trivially obtainable from a chain-class prior alone, since the bare backbone with the same context budget gives $\Delta_{\text{cb}} = 0$ by construction. *Second*, the chain-conditional framing is exactly the property that makes memres complementary to RAG rather than redundant with it: a per-fact retriever and a chain-summary compressor address orthogonal failure modes (lexical-mismatch on NoLiMa-style queries vs. context-budget exhaustion on long chains). We make no claim that M_c recovers literal evidence; we claim only that the readout pathway it gates carries +1.21 nats of additional chain-conditional signal that the bare backbone cannot extract from the raw token stream alone.

8 Ablations: which design choices are load-bearing?

Table 2 reports three single-variable ablations, each removing one component of the full recipe. All cells are at 0.6B, trained from the same seed (1) plus additional seeds where noted.

The two "positive" ablation rows (no-depth and no-floor) reduce Δ_{cb} by $\sim 20\times$ relative to the F3-off headline. The "no F3 readout probe" row is the most surprising: removing what was originally framed as the load-bearing readout-supervision channel *strengthens* the result by $\sim 8\times$. We interpret this as joint LM-NLL training producing a richer chain-conditional compression than the supervised readout shortcut; the probe pulled the readout in a direction that did not help end-to-end CE. We retain this as the canonical recipe and treat the F3-off pivot as a finding worth its own systematic study (deferred to a companion paper on the methodology of evaluating recurrent memory honestly).

9 Limitations

- **LongMemEval-S only; OOD transfer is unestablished.** The one OOD probe we ran applies the (now non-canonical) v24a checkpoint — the with-F3 recipe that Table 2 identifies as $\sim 8\times$ *worse* than the headline — to LoCoMo [15]. The result is $\Delta \approx 0$ on chain-mean CE (within bootstrap noise). We have not yet re-evaluated the canonical no-F3 (v27b) checkpoint OOD; until we do, the honest scope statement is that the present recipe trains a LongMemEval-S-domain writer and we make no transfer claim. Any rebuttal that hinges on positive OOD evidence should treat that as future work, not an established property of the method.
- **The comparison fixes the backbone family.** We use a *bare* pretrained Qwen3 backbone for RAG, so the RAG cells are not given the benefit of an instruction-tuned reading head. An instruction-tuned RAG backbone would close some of the gap. We acknowledge this as a known limitation; the strict-same-budget oracle-RAG cell is the most we can do without making the comparison uneven.
- **No same-harness recurrent-memory baselines.** We compare against RAG, not against RMT [2], Block-Recurrent [7], or AutoCompressors [3] run end-to-end on LongMemEval-S. The reason is that all three require fine-tuning the LM backbone, which would defeat the point of the present frozen-backbone comparison: the resulting cells would mix "memory

module helps" and "additional LM capacity helps" and we would not know which factor moved Δ_{cb}. A fair head-to-head against those methods requires either freezing their backbones (which prior work does not do, and which their reported recipes do not target) or unfreezing ours. Both are separate experiments worth running and we leave them to follow-up work.

- $\Delta_{\mathrm{sh}} \approx 0$ **is the project's framing finding, not its strongest claim.** A future memres recipe with a writer-side extractive objective might force chain-specific bindings into M_c and lift Δ_{sh} above noise.
- **1.7B is n=2 seeds, deliberately.** The 4-seed mean at 0.6B is the statistical headline; the 1.7B cell is scoped as a *direction-of-effect* corroboration, not a scaling claim. The two 1.7B seeds are very tight ($+0.91$, $+0.94$; per-seed $\sigma = 0.025$, $\approx 20\times$ tighter than the per-seed spread at 0.6B), which is consistent with—but does not prove—a stable scaling story; resolving that would require $n \geq 4$ at 1.7B and at least one cell at 7B+. We flag both as out-of-scope for this paper.

10 Conclusion

Recurrent memory and retrieval should be evaluated against each other on the *same eval harness*, with chain-shuffle and evidence- redacted controls. On LongMemEval-S, a small jointly-trained recurrent memory ($\sim$6 % parameter overhead, 1.5 h on a single H100) matches or exceeds strong RAG baselines including oracle-evidence retrieval, while adding zero context-token cost. The mechanism is different: recurrent memory does chain-conditional context compression; retrieval does literal evidence localisation. Both are useful; we argue they should be combined.

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. In *arXiv preprint arXiv:1607.06450*, 2016.

[2] Aydar Bulatov, Yury Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[3] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[4] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.

[5] Bradley Efron. Bootstrap methods: Another look at the jackknife. In *The Annals of Statistics*, volume 7, pages 1–26, 1979.

[6] Cheng-Ping Hsieh et al. RULER: What's the real context size of your long-context language models?, 2024.

[7] DeLesley S. Hutchins, Imanol Schlag, Yuhuai Wu, Ethan Dyer, and Behnam Neyshabur. Block-recurrent transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[8] Andrew Jaegle, Sebastian Borgeaud, Jean-Baptiste Alayrac, Carl Doersch, Catalin Ionescu, David Ding, Skanda Koppula, Daniel Zoran, Andrew Brock, Evan Shelhamer, and Olivier Hénaff. Perceiver IO: A general architecture for structured inputs and outputs. In *International Conference on Learning Representations (ICLR)*, 2022.

[9] Kimi Team. Attention residuals, 2026. https://github.com/MoonshotAI/Attention-Residuals.

[10] Alex Laitenberger et al. Stronger baselines for retrieval-augmented generation with long-context language models, 2025.

[11] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[12] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[13] Francesco Locatello, Dirk Weissenborn, Thomas Unterthiner, Aravindh Mahendran, Georg Heigold, Jakob Uszkoreit, Alexey Dosovitskiy, and Thomas Kipf. Object-centric learning with slot attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[14] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[15] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents, 2024.

[16] Ali Modarressi et al. NoLiMa: Long-context evaluation beyond literal matching, 2025.

[17] Qwen Team. Qwen3 technical report, 2025.

[18] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations (ICLR)*, 2020.

[19] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[20] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. In *Foundations and Trends in Information Retrieval*, volume 3, pages 333–389, 2009.

[21] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations (ICLR)*, 2022.

[22] Di Yu et al. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.

[23] Shuangfei Zhai, Tatiana Likhomanenko, Etai Littwin, Dan Busbridge, Jason Ramapuram, Yizhe Zhang, Jiatao Gu, and Joshua M. Susskind. Stabilizing transformer training by preventing attention entropy collapse. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

A Reproduction details

Code and data. The Memory Residuals architecture, training loop, and evaluation harness are released in the supplementary zip. The training corpus is LongMemEval-S [22], publicly available; we provide the pre-tokenised cache used by the evaluator (`paper_artifacts/chains/lme_val_s512_evpos.pt`).

Hardware. Single NVIDIA H100 80 GB per training cell. 0.6B cells finish in $\sim$1.5 h; 1.7B cells in $\sim$3–6 h (depending on the H100 SKU; we used both H100-NVL and GH200).

Hyperparameters. See Table 3.

Table 3: Training hyperparameters (identical across all cells; only seed varies).

backbone (frozen)	Qwen3-0.6B / 1.7B
K (memres slot count)	128
L_E (extraction depth)	4
D (iterative readout depth)	4
λ_{floor} (α-aux weight)	0.5
α^*_{floor} (target)	0.10
optimizer	AdamW [14], $\beta = (0.9, 0.95)$, wd 0.1
lr (memres)	1e$-$4 cosine, 200 warmup, 5 % min ratio
lr (backbone)	0 (frozen)
batch	4 chains $\times$ grad-accum 2 (effective 8) at 0.6B; 2 $\times$ 4 (effective 8) at 1.7B
TBPTT window	3 sessions
steps	1000 (0.6B) / 1500 (1.7B)
seeds reported	$\{1, 2, 3, 4\}$ at 0.6B; $\{1, 2, 3, 4\}$ at 1.7B (subject to landing)

B Per-cell numbers (raw)

Tables 4 and 5 report the per-seed and per-cell raw numbers that drive the bootstrap CIs in Table 1. All cells were evaluated on the same 50-chain validation corpus (`lme_val_s512_evpos.pt`).

Table 4: Memory Residuals per-seed numbers (Δ_{cb}, Δ_{sh}, evidence_lift) at both scales.

size / cell	seed	Δ_{cb}	Δ_{sh}	evidence_lift
0.6B v27b (no F3)	1	$+0.797$	-0.017	-0.005
0.6B v27b (no F3)	2	$+0.939$	$+0.008$	-0.002
0.6B v27b (no F3)	3	$+1.833$	$+0.000$	$+0.002$
0.6B v27b (no F3)	4	$+1.721$	$+0.001$	$+0.008$
0.6B 4-seed mean	—	$\mathbf{+1.323 \pm 0.530}$	$+0.000 \pm 0.010$	$+0.001 \pm 0.006$
1.7B v28a (no F3)	1	$+0.909$	-0.001	$+0.005$
1.7B v28b (no F3)	2	$+0.944$	-0.009	-0.008
1.7B 2-seed mean	—	$\mathbf{+0.926 \pm 0.025}$	-0.005	-0.001

Table 5: RAG baseline per-cell numbers.

size	cell	CE_{rag}	CE_{nomem}	Δ_{cb}	per-chain pos./50
0.6B	nomem (bare)	4.317	4.317	$+0.000$	0
0.6B	BM25 top-1	4.285	4.317	$+0.033$	20
0.6B	BM25 top-3	4.330	4.317	-0.012	19
0.6B	dense top-3	4.269	4.317	$+0.048$	20
0.6B	oracle top-3	4.206	4.317	$+0.111$	25
1.7B	nomem (bare)	4.703	4.703	$+0.000$	0
1.7B	BM25 top-3	4.450	4.703	$+0.253$	29
1.7B	dense top-3	4.668	4.703	$+0.035$	32
1.7B	oracle top-3	4.315	4.703	$+0.388$	31

C Bootstrap CI methodology

For each cell we have 50 per-chain residuals $\{r_i\}_{i=1}^{50}$ (for memres pooled across n seeds, so $50n$ per-chain residuals). We resample with replacement at the chain level $B = 5000$ times, compute the mean of each resample, and report the $[2.5, 97.5]$ percentiles as the 95 % CI. This is the standard bootstrap of [5]; the cluster-by-chain choice respects the fact that within-chain residuals are correlated (different methods score the same callback positions).

NeurIPS Paper Checklist

1. **Claims**
   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?
   Answer: `[Yes]`
   Justification: Each numbered contribution in Section 1 is reported in Section 6–8 with a specific number, seed count, CI, and the matched RAG baseline that grounds the comparison. Scope statements (no LoCoMo SoTA claim, no instruction- tuned RAG backbone) appear in Section 9.

2. **Limitations**
   Question: Does the paper discuss the limitations of the work?
   Answer: `[Yes]`
   Justification: Section 9 dedicates a half-page to four explicit limitations: domain specificity, the bare-backbone RAG framing, the chain-shuffle $\approx$0 framing finding, and the n=2 1.7B seed count.

3. **Theory assumptions and proofs**
   Answer: `[N/A]`
   Justification: The paper is empirical; no formal theorems are stated.

4. **Experimental result reproducibility**
   Answer: `[Yes]`
   Justification: Section 3.4 and Appendix A document hardware, hyperparameters, the training corpus, the eval corpus, and the eval script. Per-seed launch scripts are released in the supplementary.

5. **Open access to data and code**
   Answer: `[Yes]`
   Justification: The supplementary zip (`memres_supplementary.zip`) contains the architecture implementation, the training loop, the launchers, and the eval harness. The training/eval corpora are derived from the publicly available LongMemEval-S [22]; we ship the pre-tokenised cache.

6. **Experimental setting / details**
   Answer: `[Yes]`
   Justification: Section 3.4 and Table 3.

7. **Experiment statistical significance**
   Answer: `[Yes]`
   Justification: Headline is reported with both seed std (± 0.53) and 95 % bootstrap-over-chains CI ($[+1.11, +1.55]$). Per-chain positive count (48/50) and median per-chain $\Delta = +0.91$ are also reported (Section 6).

8. **Experiments compute resources**
   Answer: `[Yes]`
   Justification: Each cell reports its training time, hardware, and parameter overhead in Section 3.4, Section 6, and Appendix A.

9. **Code of ethics**
   Answer: `[Yes]`
   Justification: The paper studies a small architectural modification to a pretrained LLM; it does not introduce new training data, does not de-anonymise user data, and does not train on copyrighted material beyond what is already in the publicly available LongMemEval-S.

We declare LLM use during paper writing and code generation in the LLM-usage field of the submission form (per the 2026 chairs' explicit request).

10. **Broader impacts**
    Answer: [Yes]
    Justification: Long-horizon memory is a building block for agentic LLM products. We add a fixed-cost differentiable alternative to context-extension and retrieval. The most plausible misuse is using the resulting agent for surveillance of conversational partners; we discuss this briefly in the supplementary README. The training procedure does not amplify any new safety risk relative to the baseline of fine-tuning the same backbone on the same data.
11. **Safeguards**
    Answer: [N/A]
    Justification: We do not release new datasets, new checkpoints, or scraped data; the released artifacts are architecture code and training scripts.
12. **Licenses for existing assets**
    Answer: [Yes]
    Justification: Qwen3 is released under Apache-2.0; LongMemEval-S is released under MIT. Both are cited.
13. **New assets**
    Answer: [Yes]
    Justification: The supplementary code is released under Apache-2.0, anonymised for the review period.
14. **Crowdsourcing and research with human subjects**
    Answer: [N/A]
15. **Institutional review board (IRB) approvals or equivalent**
    Answer: [N/A]
16. **Declaration of LLM usage**
    Answer: [Yes]
    Justification: An LLM coding assistant (Cursor agent based on Anthropic Claude) was used throughout the project for code generation, experiment scripting, run-log analysis, and manuscript drafting; all numbers in the paper come from agent-launched but human-verified runs against the locked corpus and eval script.